STM32F411 Mini Oscilloscope

with Integrated PWM Function Generator

Technical Documentation

Firmware Source Code Reference – v1.0

Platform: STM32F411CEU6 (BlackPill)
Display: ILI9341 2.8" TFT (SPI)
IDE: STM32CubeIDE **HAL:** STM32F4xx HAL
Language: C (ISO C99)
Year: 2026

Project Team

Jash Dhokiya	23BIT058
Kavisha Sharma	23BIT067
Dhyeya Choksi	23BIT055
Tanay Shah	23BIT062

Pandit Deendayal Energy University
Department of ICT / ECE
Embedded Systems Laboratory

Copyright © 2026. Generated documentation. All rights reserved.

Contents

1 Introduction **3**
1.1 Project Overview . . . 3
1.2 Feature Summary . . . 3
1.3 Repository File Map . . . 3

2 Hardware Configuration **5**
2.1 Microcontroller . . . 5
2.1.1 System Clock Configuration . . . 5
2.1.2 Bus Clocks . . . 5
2.2 GPIO Pin Assignments . . . 5
2.3 ADC Configuration . . . 6
2.4 Timer Summary . . . 7

3 Firmware Architecture **8**
3.1 Layer Overview . . . 8
3.2 Main Loop Logic . . . 8
3.3 Interrupt Priority Table . . . 9

4 Module Reference **10**
4.1 `osc_adc.c` — ADC Application Layer . . . 10
4.1.1 Purpose . . . 10
4.1.2 Buffers . . . 10
4.1.3 API . . . 10
4.1.4 De-interleave Algorithm . . . 10
4.1.5 Callbacks . . . 11
4.2 `osc_signal.c` — Signal Processing . . . 11
4.2.1 Purpose . . . 11
4.2.2 Function Reference . . . 11
4.2.3 Frequency Measurement Algorithm . . . 11
4.2.4 Voltage Conversion . . . 12
4.2.5 Pixel Mapping . . . 12
4.3 `osc_display.c` — Display Rendering . . . 12
4.3.1 Purpose . . . 12
4.3.2 Display Layout . . . 12
4.3.3 API . . . 12
4.3.4 Waveform Rendering Detail . . . 13
4.4 `osc_ui.c` — User Interface State Machine . . . 13

4.4.1 State Machine . . . 13
4.4.2 Button Mapping . . . 13
4.4.3 Debounce . . . 13
4.5 `func_gen.c` — PWM Function Generator . . . 13
4.5.1 Architecture . . . 14
4.5.2 Waveform LUTs . . . 14
4.5.3 Frequency Formula . . . 14
4.5.4 ISR – `FuncGen_StepISR()` . . . 14
4.5.5 API . . . 14
4.6 `stm32f4xx_it.c` — Interrupt Service Routines . . . 15
4.6.1 ISR Routing Table . . . 15

5 Data Flow and Timing **16**
5.1 Oscilloscope Data Flow . . . 16
5.2 Sample Rate Calculation . . . 16
5.3 Function Generator Timing . . . 16

6 Configuration Reference **17**

7 Build and Flash Instructions **18**
7.1 Prerequisites . . . 18
7.2 Project Setup . . . 18
7.3 Expected Boot Sequence . . . 18

8 Limitations and Future Work **19**
8.1 Current Limitations . . . 19
8.2 Suggested Improvements . . . 19

A Interrupt Vector Summary **20**

B Colour Palette **21**

C Glossary **22**

1. Introduction

1.1 Project Overview

This project implements a dual-channel digital oscilloscope combined with a multi-waveform PWM function generator, running entirely on an STM32F411CEU6 (*BlackPill*) microcontroller. The system samples two analog input channels using the on-chip 12-bit ADC with DMA, displays real-time waveforms on an ILI9341 TFT LCD over SPI, computes signal parameters (frequency and V_{pp}), and simultaneously produces a user-selectable waveform on GPIO pin PB8 via PWM with an external RC low-pass filter.

The firmware is structured into well-separated modules: HAL-generated peripheral drivers (ADC, DMA, SPI, TIM, GPIO, USART), and custom application layers for ADC management, signal processing, display rendering, UI/button handling, and function generation.

1.2 Feature Summary

Core Features

- **Dual-channel ADC input:** CH1 (PA0) and CH2 (PA1), 12-bit resolution, timer-triggered via TIM2
- **DMA transfer:** Circular mode on DMA2 Stream 0 – zero CPU overhead for sampling
- **Configurable time base:** Multiple time/division settings via TIM2 auto-reload register
- **Signal measurements:** Real-time frequency (zero-crossing method) and V_{pp}
- **ILI9341 display:** Waveform, grid, measurements, and status bar over SPI1
- **Trigger system:** Rising-edge trigger search per acquired buffer
- **RUN / HOLD mode:** Toggle via push-button (PB0)
- **Overall stats overlay:** Comprehensive signal statistics (freq, V_{pp}, V_{min}, V_{max}, V_{avg}) via push-button (PB4)
- **Function generator:** Sine, square, triangle, sawtooth – LUT-based PWM on PB8 (TIM4 CH3)
- **Analog watchdog:** ADC overvoltage protection (threshold = 3700/4095)

1.3 Repository File Map

File	Category	Responsibility
`main.c`	Application	System init, splash screen, main loop
`adc.c`	HAL/Peripheral	ADC1 configuration, DMA linkage, GPIO analog pins
`dma.c`	HAL/Peripheral	DMA2 clock enable, interrupt priority
`spi.c`	HAL/Peripheral	SPI1 master config for ILI9341
`tim.c`	HAL/Peripheral	TIM2 (ADC trigger), TIM3 (input capture), TIM4 (PWM), TIM5 (FG step)
`gpio.c`	HAL/Peripheral	Output pins, button EXTI lines
`usart.c`	HAL/Peripheral	USART1 (optional debug)
`osc_adc.c`	Application	ADC start/stop, buffer de-interleave, DMA callback
`osc_signal.c`	Application	Frequency, V_{pp}, trigger, pixel mapping
`osc_display.c`	Application	Grid, waveform, measurement bar rendering
`osc_ui.c`	Application	Button debounce, state machine, index cycling
`func_gen.c`	Application	LUT tables, TIM4/TIM5 PWM function generator
`stm32f4xx_it.c`	ISR	All ISR vectors, EXTI/TIM5/DMA/ADC callbacks
`stm32f4xx_hal_msp.c`	HAL/Peripheral	MSP init/deinit callbacks
`syscalls.c`	System	Newlib syscall stubs
`sysmem.c`	System	Heap memory implementation
`system_stm32f4xx.c`	CMSIS	SystemClock and SystemCoreClock init

2. Hardware Configuration

2.1 Microcontroller

The STM32F411CEU6 (BlackPill) is a Cortex-M4F running at up to 100 MHz. In this project the PLL is configured to deliver exactly **100 MHz** system clock from a 25 MHz HSE crystal, enabling deterministic sample rate calculations.

2.1.1 System Clock Configuration

```
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
RCC_OscInitStruct.HSEState       = RCC_HSE_ON;
RCC_OscInitStruct.PLL.PLLState   = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource  = RCC_PLLSOURCE_HSE;
RCC_OscInitStruct.PLL.PLLM       = 25;    /* divide 25 MHz -> 1 MHz */
RCC_OscInitStruct.PLL.PLLN       = 200;   /* x200 -> 200 MHz VCO    */
RCC_OscInitStruct.PLL.PLLP       = RCC_PLLP_DIV2;  /* /2 -> 100 MHz */
RCC_OscInitStruct.PLL.PLLQ       = 4;
```

Listing 2.1: PLL configuration in `SystemClock_Config()`

$$f_{SYSCLK} = \frac{f_{HSE} \times N}{M \times P} = \frac{25 \times 200}{25 \times 2} = \mathbf{100}\,\text{MHz}$$

2.1.2 Bus Clocks

Bus	Divider	Frequency
SYSCLK	1	100 MHz
HCLK (AHB)	1	100 MHz
PCLK1 (APB1)	2	50 MHz
PCLK2 (APB2)	1	100 MHz

Flash Latency

Flash latency is set to `FLASH_LATENCY_3` (3 wait states) as required for 100 MHz Vcore Scale 1.

2.2 GPIO Pin Assignments

Port	Pin	Function	Description
PA0	–	ADC1_IN0	Channel 1 analog input
PA1	–	ADC1_IN1	Channel 2 analog input
PA3	–	GPIO Output	ILI9341 DC (Data/Command)
PA4	–	GPIO Output	ILI9341 CS (Chip Select)
PA5	–	SPI1_SCK	ILI9341 SPI clock
PA6	–	TIM3_CH1	Input capture (reserved/debug)
PA7	–	SPI1_MOSI	ILI9341 SPI data
PB0	–	EXTI0 (rising)	Button: RUN/HOLD toggle
PB1	–	EXTI1 (rising)	Button: Time div cycle
PB2	–	EXTI2 (rising)	Button: Volt div cycle
PB3	–	EXTI3 (rising)	Button: Channel toggle
PB4	–	EXTI4 (rising)	Button: Show overall signal statistics
PB6	–	GPIO Output	ILI9341 RST
PB7	–	GPIO Output	ILI9341 LED backlight
PB8	–	TIM4_CH3	Function generator PWM output

2.3 ADC Configuration

ADC1 is configured in `adc.c` with the following parameters:

Parameter	Value
Resolution	12-bit
Clock prescaler	PCLK2 / 4 = 25 MHz ADC clock
Scan mode	Enabled (2 channels)
Conversion mode	Discontinuous (timer-triggered)
External trigger	TIM2 TRGO (rising edge)
DMA mode	Circular, half-word
Analog watchdog	All regular channels, High = 3700

Both channels are ranked in sequence (Rank 1 = CH0/PA0, Rank 2 = CH1/PA1), with 15 ADC clock cycle sampling time each. The interleaved raw buffer layout is:

$$\text{adcRawBuffer} = [\underbrace{CH1}_{0}, \underbrace{CH2}_{1}, \underbrace{CH1}_{2}, \underbrace{CH2}_{3}, \ldots]$$

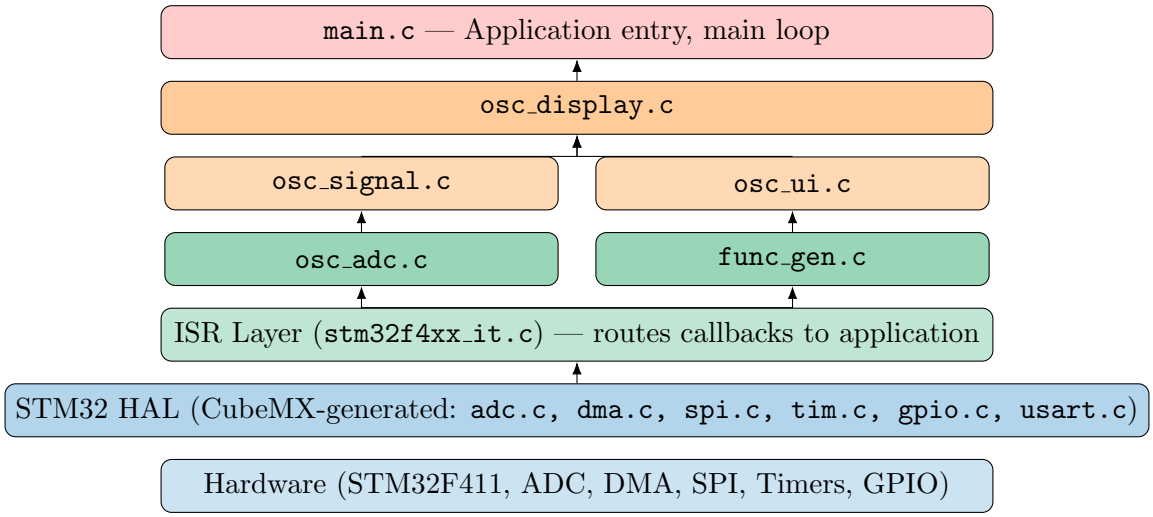
2.4 Timer Summary

Timer	Mode	Period	Purpose
TIM2	Base + TRGO	Runtime variable (ARR)	ADC sample rate trigger; ARR = $\frac{100\,\text{MHz}}{f_s} - 1$
TIM3	Input Capture	65535	CH1 on PA6 (reserved for HW frequency measurement)
TIM4	PWM CH3	`FUNCGEN_PWM_ARR`	High-frequency carrier for function generator
TIM5	Base + IT	$\frac{f_{clk}}{f_{out} \times 64} - 1$	LUT step interrupt for function generator

3. Firmware Architecture

3.1 Layer Overview

The firmware follows a layered architecture separating HAL-generated peripheral code from user application modules.



3.2 Main Loop Logic

After peripheral initialization and the splash screen, `main()` enters an infinite loop that polls a `volatile` flag `bufferReady` (set by the DMA completion ISR) and the UI state machine:

```
while (1) {
    if (bufferReady && UI_GetState() == STATE_RUN) {
        bufferReady = 0;
        uint16_t* buf = OSC_ADC_GetBuffer(UI_GetChannel());
        float freq = Signal_GetFrequency(buf, SAMPLE_BUFFER_SIZE,
    sampleRate);
        float vpp  = Signal_GetVpp(buf, SAMPLE_BUFFER_SIZE);

        Display_ClearWaveform();
        Display_DrawGrid();
        Display_DrawWaveform(buf, SAMPLE_BUFFER_SIZE, channel, voltDiv);
        Display_DrawMeasurements(freq, vpp, channel, timeDivIdx,
    voltDivIdx, 0);
    }
    else if (UI_GetState() == STATE_HOLD) {
```

```
        /* Redraw measurements once with HOLD label */
    }
}
```

Listing 3.1: Main application loop (simplified from `main.c`)

Design Note

The main loop is **interrupt-driven** – it never polls the ADC directly. All sampling is triggered by TIM2 → ADC → DMA. The CPU is only involved when `bufferReady = 1` is set from the DMA ISR, making it quasi-event-driven without an RTOS.

3.3 Interrupt Priority Table

Interrupt	Preempt	Sub	Function
DMA2_Stream0	0	0	ADC DMA transfer complete
ADC (analog watchdog)	0	0	Overvoltage stop
TIM5	2	0	Function generator LUT step
EXTI0–EXTI3	3	0	Button press handling

4. Module Reference

4.1 osc_adc.c — ADC Application Layer

4.1.1 Purpose

Wraps the CubeMX-generated `hadc1` and `htim2` handles into a clean start/stop API and exposes per-channel sample buffers. It also performs de-interleaving of the raw DMA buffer.

4.1.2 Buffers

```c
static uint16_t adcRawBuffer[ADC_BUFFER_SIZE]; /* interleaved CH1,CH2,... */
static uint16_t ch1Buffer[SAMPLE_BUFFER_SIZE];
static uint16_t ch2Buffer[SAMPLE_BUFFER_SIZE];
volatile uint8_t bufferReady = 0;
```

Listing 4.1: Buffer declarations in `osc_adc.c`

`ADC_BUFFER_SIZE = 2 × SAMPLE_BUFFER_SIZE` to hold both channels interleaved.

4.1.3 API

Function	Returns	Description
`OSC_ADC_Start()`	void	Starts ADC DMA + TIM2
`OSC_ADC_Stop()`	void	Stops TIM2 + ADC DMA
`OSC_ADC_GetBuffer(ch)`	`uint16_t*`	Returns pointer to de-interleaved channel buffer
`OSC_ADC_SetSampleRate(arr)`	void	Dynamically updates TIM2 ARR

4.1.4 De-interleave Algorithm

```c
static void DeInterleave(void) {
    for (uint16_t i = 0; i < SAMPLE_BUFFER_SIZE; i++) {
        ch1Buffer[i] = adcRawBuffer[i * 2];
        ch2Buffer[i] = adcRawBuffer[i * 2 + 1];
    }
}
```

Listing 4.2: De-interleave routine

4.1.5 Callbacks

`HAL_ADC_ConvCpltCallback` Called by the DMA ISR when the full buffer is transferred. Sets `bufferReady = 1`.

`HAL_ADC_LevelOutOfWindowCallback` Called by the ADC analog watchdog ISR when input exceeds 3700/4095 ($\approx 3.0\,\mathrm{V}$). Calls `OSC_ADC_Stop()` to protect inputs.

4.2 `osc_signal.c` — Signal Processing

4.2.1 Purpose

Pure computation module. Takes raw 12-bit ADC buffers and returns physical measurements. No HAL calls.

4.2.2 Function Reference

Function	Returns	Description
`Signal_GetFrequency(buf, size, sr)`	`float`	Frequency via rising zero-crossing count
`Signal_GetVpp(buf, size)`	`float`	Peak-to-peak voltage in volts
`Signal_GetVmin(buf, size)`	`float`	Minimum voltage
`Signal_GetVmax(buf, size)`	`float`	Maximum voltage
`Signal_GetAvg(buf, size)`	`float`	DC average voltage
`Signal_FindTriggerPoint(buf, size, thresh)`	`uint16_t`	Index of first rising edge crossing `thresh`
`Signal_MapToPixel(adcVal, voltDiv)`	`uint16_t`	Maps ADC code to display Y pixel

4.2.3 Frequency Measurement Algorithm

The frequency is estimated from the number of rising zero-crossings across the buffer. The midpoint threshold is auto-computed from V_{min} and V_{max}:

```
uint16_t midpoint = (vMin + vMax) / 2;
if ((vMax - vMin) < 50) return 0.0f;  /* signal too weak */

for (uint16_t i = 1; i < size; i++) {
    if (buffer[i-1] < midpoint && buffer[i] >= midpoint) {
        crossingCount++;
        if (crossingCount == 1) firstCrossing = i;
        lastCrossing = i;
    }
}
float samplesPerPeriod = (lastCrossing - firstCrossing) / (crossingCount
    - 1);
return (float)sampleRate / samplesPerPeriod;
```

Listing 4.3: Zero-crossing frequency algorithm

4.2.4 Voltage Conversion

$$V_{pp} = \frac{ADC_{max} - ADC_{min}}{4095} \times V_{REF} \times R_{div}$$

where $V_{REF} = 3.3\,\mathrm{V}$ and R_{div} is the input voltage divider ratio defined in `config.h`.

4.2.5 Pixel Mapping

$$y_{pixel} = y_{center} - \left(\frac{ADC_{val} - 2048}{ADC_{per_div}}\right) \times \frac{H_{wave}}{V_{divs}}$$

Clamped to $[WAVE_Y_OFFSET,\ WAVE_Y_OFFSET + WAVE_HEIGHT - 1]$.

4.3 `osc_display.c` — Display Rendering

4.3.1 Purpose

Manages all ILI9341 rendering: initialization, grid drawing, waveform plotting with vertical line interpolation between samples, and the top/bottom HUD bars.

4.3.2 Display Layout

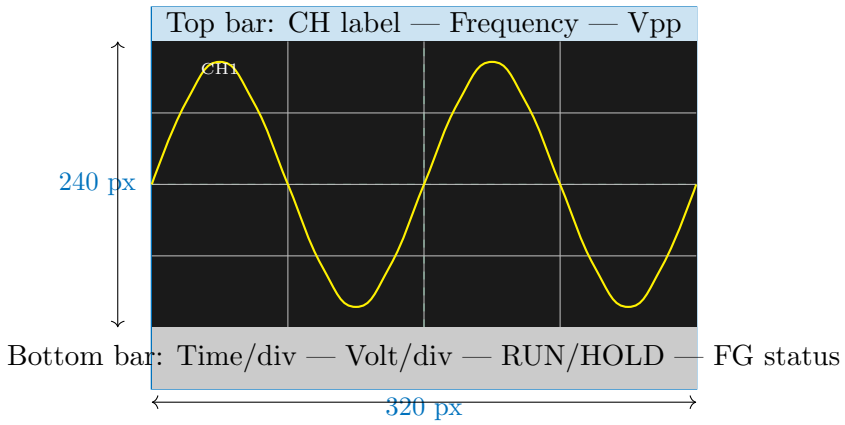


Figure 4.1: ILI9341 display layout (320×240 pixels)

4.3.3 API

Function	Returns	Description
`Display_Init()`	void	Init ILI9341, fill black, reset prev-wave state
`Display_ClearScreen()`	void	Fill entire screen black
`Display_ClearWaveform()`	void	Clear waveform region only
`Display_DrawGrid()`	void	Dotted grid + dashed centre axes
`Display_DrawWaveform(buf,sz,chvid)`	void	Trigger-aligned waveform with line interpolation
`Display_DrawMeasurements(...)`	void	Top bar (freq, Vpp), bottom bar (T/div, V/div, mode, FG)

Function	Returns	Description
`Display_ShowWarning(msg)`	void	Centred red warning overlay box

4.3.4 Waveform Rendering Detail

The waveform is rendered starting from the **trigger point** (first rising edge). For each sample pair $(i-1, i)$, a pixel and a connecting line are drawn to avoid gaps at low time/div settings:

```
ILI9341_DrawPixel(x, y, waveColor);
if (i > 0)
    ILI9341_DrawLine(WAVE_X_OFFSET + i - 1, prevY, x, y, waveColor);
```

Listing 4.4: Line interpolation in waveform rendering

4.4 `osc_ui.c` — User Interface State Machine

4.4.1 State Machine

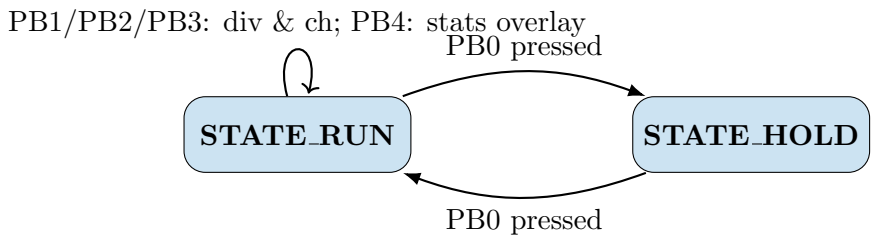


Figure 4.2: UI state machine

4.4.2 Button Mapping

GPIO Pin	EXTI	Action
PB0	EXTI0	Toggle RUN ↔ HOLD; also stops/restarts ADC
PB1	EXTI1	Cycle time/div index (0 → NUM_TIME_DIVS −1) + update TIM2 ARR
PB2	EXTI2	Cycle volt/div index (0 → NUM_VOLT_DIVS −1)
PB3	EXTI3	Toggle active channel (CH1 ↔ CH2)
PB4	EXTI4	Display overall signal statistics overlay (freq, V_{pp}, V_{min}, V_{max}, V_{avg})

4.4.3 Debounce

Software debounce of `BUTTON_DEBOUNCE_MS` milliseconds is applied using `HAL_GetTick()` at the start of `UI_HandleButton()`.

4.5 `func_gen.c` — PWM Function Generator

4.5.1 Architecture

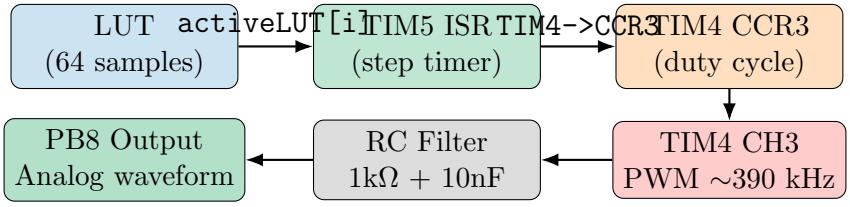


Figure 4.3: Function generator signal chain

4.5.2 Waveform LUTs

Four 64-sample lookup tables are stored in `static const uint8_t` arrays (values 0–255):

Waveform	Table	Description
Sine	`sineLUT[64]`	Full-period sinusoid
Square	`squareLUT[64]`	50% duty, 255 for first 32 samples, 0 for last 32
Triangle	`triangleLUT[64]`	Linear ramp up + down
Sawtooth	`sawtoothLUT[64]`	Linear ramp 0 → 255

4.5.3 Frequency Formula

TIM5 auto-reload register (ARR) controls the step rate:

$$ARR_{TIM5} = \frac{f_{clk}}{f_{out} \times N_{LUT}} - 1 = \frac{100\,\text{MHz}}{f_{out} \times 64} - 1$$

$$f_{PWMcarrier} \approx \frac{f_{clk}}{ARR_{TIM4} + 1} \approx 390\,\text{kHz}$$

The RC filter cut-off frequency is:

$$f_c = \frac{1}{2\pi RC} = \frac{1}{2\pi \times 1000 \times 10 \times 10^{-9}} \approx 15.9\,\text{kHz}$$

4.5.4 ISR – `FuncGen_StepISR()`

Called directly from `TIM5_IRQHandler` (using raw register access to minimise latency):

```
void FuncGen_StepISR(void) {
    TIM4->CCR3 = activeLUT[lutIndex];
    lutIndex = (lutIndex + 1) & (FUNCGEN_LUT_SIZE - 1);
}
```

Listing 4.5: LUT step ISR (latency-critical)

The bitmask `& 63` wraps the index without a branch.

4.5.5 API

Function	Returns	Description
`FuncGen_Init()`	void	Sets defaults (sine, 1 kHz), resets state
`FuncGen_Start()`	void	Starts TIM4 PWM + TIM5 step timer
`FuncGen_Stop()`	void	Stops TIM5, zeros duty, stops TIM4
`FuncGen_SetWaveform(type)`	void	Switches active LUT pointer
`FuncGen_SetFrequency(Hz)`	void	Updates TIM5 ARR dynamically
`FuncGen_IsRunning()`	`uint8_t`	Returns running flag
`FuncGen_GetFrequency()`	`uint32_t`	Returns current set frequency
`FuncGen_GetWaveformName()`	`const char*`	Returns "SIN", "SQR", "TRI", "SAW"

4.6 `stm32f4xx_it.c` — Interrupt Service Routines

4.6.1 ISR Routing Table

ISR	Action
`SysTick_Handler`	`HAL_IncTick()` – millisecond tick for HAL timeouts and debounce
`DMA2_Stream0_IRQHandler`	`HAL_DMA_IRQHandler()` → triggers `HAL_ADC_ConvCpltCallback`
`ADC_IRQHandler`	`HAL_ADC_IRQHandler()` → analog watchdog callback
`EXTI0...3_IRQHandler`	`HAL_GPIO_EXTI_IRQHandler()` → `UI_HandleButton(pin)`
`TIM5_IRQHandler`	Direct register: clears UIF, calls `FuncGen_StepISR()`
`HardFault_Handler`	Infinite loop (breakpoint target)

Why TIM5 bypasses HAL

`TIM5_IRQHandler` directly clears `TIM5->SR` and calls `FuncGen_StepISR()` without going through `HAL_TIM_IRQHandler()`. This avoids ∼20 HAL overhead instructions, which matters at high function generator frequencies where TIM5 fires at up to $f_{out} \times 64$ times per second.

5. Data Flow and Timing

5.1 Oscilloscope Data Flow

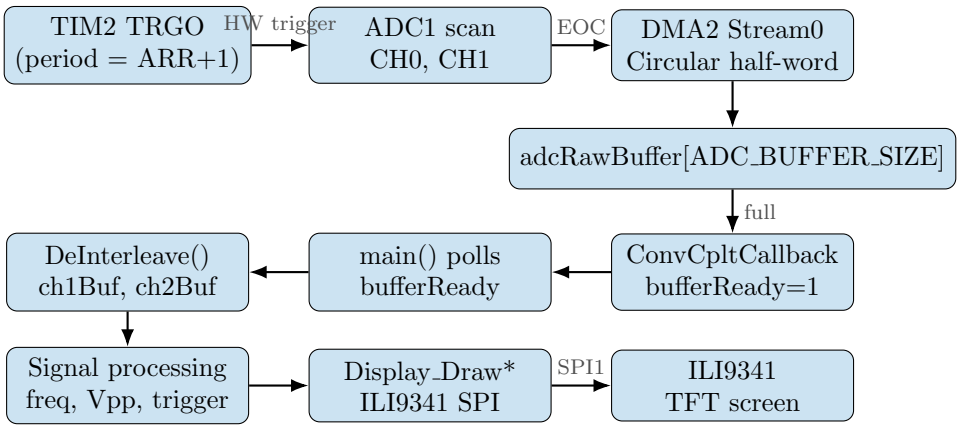


Figure 5.1: End-to-end oscilloscope data flow

5.2 Sample Rate Calculation

TIM2 runs on APB1 $\times 2 = 100\,\text{MHz}$. The sample rate in samples per second *per channel* is:

$$f_s = \frac{100\,\text{MHz}}{(ARR + 1) \times 2}$$

The factor of 2 accounts for two channels being scanned per TIM2 trigger. The `TIME_DIV_ARR[]` lookup table in `config.h` maps time/div settings to the correct ARR value.

5.3 Function Generator Timing

TIM5 fires at $f_{step} = f_{out} \times 64$. At the default frequency of 1 kHz:

$$ARR_{TIM5} = \frac{100\,\text{MHz}}{1000 \times 64} - 1 = 1562 - 1 = 1561$$

The PWM carrier on TIM4 at $\approx 390\,\text{kHz}$ combined with the 15.9 kHz RC filter produces a smooth analog approximation of the selected waveform.

6. Configuration Reference

All project-wide constants are consolidated in `config.h`. Key parameters:

Macro	Typical Value	Description
SAMPLE_BUFFER_SIZE	512	Samples per channel per acquisition
ADC_BUFFER_SIZE	1024	Raw DMA buffer (2× above)
ADC_MAX	4095	12-bit full-scale
VREF	3.3f	ADC reference voltage (V)
VOLTAGE_DIVIDER_RATIO	≥1.0	Input resistor divider scale
SCREEN_WIDTH	320	ILI9341 pixel width
SCREEN_HEIGHT	240	ILI9341 pixel height
WAVE_X_OFFSET	–	Left edge of waveform area
WAVE_Y_OFFSET	–	Top edge of waveform area
WAVE_WIDTH	–	Waveform area width
WAVE_HEIGHT	–	Waveform area height
GRID_DIVISIONS_X	8	Horizontal grid divisions
GRID_DIVISIONS_Y	6	Vertical grid divisions
NUM_TIME_DIVS	–	Length of time-div array
NUM_VOLT_DIVS	–	Length of volt-div array
NUM_CHANNELS	2	Oscilloscope channels
BUTTON_DEBOUNCE_MS	50	Button debounce time (ms)
FUNCGEN_DEFAULT_FREQ	1000	Default function gen frequency (Hz)
FUNCGEN_LUT_SIZE	64	Samples per LUT waveform period
FUNCGEN_PWM_ARR	≈255	TIM4 ARR for PWM carrier
FUNCGEN_TIM_CLOCK	100000000	TIM5 clock frequency (Hz)

7. Build and Flash Instructions

7.1 Prerequisites

- STM32CubeIDE $\geq$ 1.14 (includes `arm-none-eabi-gcc`)
- STM32F4xx HAL library (included in CubeIDE)
- ST-Link V2 programmer, or DFU via USB

7.2 Project Setup

1. Open STM32CubeIDE and import or create an STM32F411CE project.
2. In CubeMX (inside IDE), configure peripherals to match Chapter 2.
3. Copy all `.c` files from `Src/` into the project's `Core/Src/` folder.
4. Ensure the following driver/library files are present (generated by CubeMX):
   - `Drivers/STM32F4xx_HAL_Driver/`
   - `Drivers/CMSIS/`
   - `Core/Inc/` headers: `config.h, osc_adc.h, osc_signal.h, osc_display.h, osc_ui.h, func_gen.h, ili9341.h`
5. Build with `Release` configuration for optimal performance.
6. Flash via Run $\rightarrow$ Debug or `ST-LINK/V2` programmer.

7.3 Expected Boot Sequence

1. HAL, system clock, and all peripherals initialize.
2. Splash screen shown for 1.5 s: "OSCILLOSCOPE / Dual Channel / + Function Generator".
3. Function generator starts (1 kHz sine on PB8).
4. ADC + DMA + TIM2 start.
5. Main loop begins: waveform acquisition and display.

8. Limitations and Future Work

8.1 Current Limitations

Known Issues

- **ADC analog watchdog:** Stop on overvoltage does not display a warning in current code – `Display_ShowWarning()` call is commented out in the callback.
- **Frequency measurement:** Zero-crossing algorithm is susceptible to noise on near-DC or irregular signals. No averaging is applied.
- **Function generator amplitude:** Fixed to 3.3 V swing (PWM × RC). No software amplitude control.
- **No RTOS:** Display latency under heavy SPI loads may cause missed ADC buffer cycles (frame dropping, not data loss).

8.2 Suggested Improvements

Enhancement Roadmap

- Add UART serial output for waveform data export.
- Implement averaging for frequency and Vpp (running mean over N frames).
- Use TIM3 input capture for hardware frequency measurement crosscheck.
- Add FreeRTOS task separation between acquisition and rendering.
- Implement function generator amplitude control via DAC or digital pot.
- Add waveform cursors (time-delta, voltage-delta measurement).
- Support FFT spectrum display using CMSIS DSP library.

A. Interrupt Vector Summary

Vector	Priority (Pre/Sub)	Handler
DMA2_Stream0_IRQn	0 / 0	`HAL_DMA_IRQHandler`
ADC_IRQn	0 / 0	`HAL_ADC_IRQHandler`
TIM5_IRQn	2 / 0	`FuncGen_StepISR` (direct)
EXTI0_IRQn	3 / 0	`UI_HandleButton(PIN_0)`
EXTI1_IRQn	3 / 0	`UI_HandleButton(PIN_1)`
EXTI2_IRQn	3 / 0	`UI_HandleButton(PIN_2)`
EXTI3_IRQn	3 / 0	`UI_HandleButton(PIN_3)`
EXTI4_IRQn	3 / 0	`UI_HandleButton(PIN_4)`
SysTick	–	`HAL_IncTick`

B. Colour Palette

The display uses the following colour definitions (RGB565 format expected by ILI9341 driver):

Macro	Usage
`COLOR_BACKGROUND`	Screen fill (black)
`COLOR_GRID`	Dotted grid lines
`COLOR_DARK_GREEN`	Centre axis lines
`COLOR_DARK_GRAY`	Waveform area border
`COLOR_CH1`	Channel 1 waveform (yellow)
`COLOR_CH2`	Channel 2 waveform (cyan)
`COLOR_GREEN`	T/div label, RUN indicator
`COLOR_RED`	HOLD indicator, warnings
`COLOR_ORANGE`	Function generator status
`COLOR_TEXT`	General measurement text
`COLOR_WHITE`	Warning overlay text

C. Glossary

ADC Analog-to-Digital Converter

ARR Auto-Reload Register (sets timer period)

CCR Capture/Compare Register (sets PWM duty cycle)

DMA Direct Memory Access

EXTI External Interrupt line

HAL Hardware Abstraction Layer (STM32 middleware)

HSE High-Speed External crystal oscillator

ILI9341 262K-colour TFT LCD controller by Ilitek

ISR Interrupt Service Routine

LUT Lookup Table

NVIC Nested Vectored Interrupt Controller

PLL Phase-Locked Loop (clock multiplier)

PWM Pulse-Width Modulation

RC filter Resistor-Capacitor low-pass filter

SPI Serial Peripheral Interface

TIM General-Purpose Timer (STM32)

TRGO Timer Trigger Output (used to trigger ADC)

$\boldsymbol{V_{pp}}$ Peak-to-peak voltage

VREF ADC reference voltage